Rekurzivní třídící algoritmy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

09.04.2026

CC BY-NC-SA 4.0

Rozděl a panuj

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj (divide and conquer)

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj (divide and conquer) – programovací paradigma, které

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj (divide and conquer) – programovací paradigma, které rozděluje problém na **několik menších** podproblémů, které je jednodušší vyřešit

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj (divide and conquer) – programovací paradigma, které rozděluje problém na **několik menších** podproblémů, které je jednodušší vyřešit

Povahově to bývají

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj (divide and conquer) – programovací paradigma, které rozděluje problém na **několik menších** podproblémů, které je jednodušší vyřešit

Povahově to bývají **rekurzivní** algoritmy

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj (divide and conquer) – programovací paradigma, které rozděluje problém na **několik menších** podproblémů, které je jednodušší vyřešit

Povahově to bývají **rekurzivní** algoritmy

Zároveň je nutné do časové složitosti zakomponovat

Rozděl a panuj

Jak byste řešili problém, který je příliš velký na to, abyste ho vyřešili najednou?

Rozděl a panuj (divide and conquer) – programovací paradigma, které rozděluje problém na **několik menších** podproblémů, které je jednodušší vyřešit

Povahově to bývají **rekurzivní** algoritmy

Zároveň je nutné do časové složitosti zakomponovat **režii pro spojení** výsledků podproblémů

Quick Sort

Princip

Princip

Quick Sort používá strategii **rozděl a panuj** (divide and conquer)

Princip

Quick Sort používá strategii **rozděl a panuj** (divide and conquer)

Postup:

1. Zvol **pivot** (referenční prvek)

Princip

Quick Sort používá strategii **rozděl a panuj** (divide and conquer)

Postup:

1. Zvol **pivot** (referenční prvek)
2. **Rozděl** (partition) pole na dvě části:

Princip

Quick Sort používá strategii **rozděl a panuj** (divide and conquer)

Postup:

1. Zvol **pivot** (referenční prvek)
2. **Rozděl** (partition) pole na dvě části:
   - Prvky **menší** než pivot → levá část
   - Prvky **větší** než pivot → pravá část

Princip

Quick Sort používá strategii **rozděl a panuj** (divide and conquer)

Postup:

1. Zvol **pivot** (referenční prvek)
2. **Rozděl** (partition) pole na dvě části:
   - Prvky **menší** než pivot → levá část
   - Prvky **větší** než pivot → pravá část
3. **Rekurzivně** seřaď obě části

Příklad – Quick Sort

Pole: `[5, 3, 8, 1, 2]`, pivot = poslední prvek

Příklad – Quick Sort

Pole: `[5, 3, 8, 1, 2]`, pivot = poslední prvek

1. úroveň: pivot = **2**

Příklad – Quick Sort

Pole: `[5, 3, 8, 1, 2]`, pivot = poslední prvek

1. úroveň: pivot = **2**

< pivot	pivot	> pivot
`[1]`	**2**	`[5, 3, 8]`

Příklad – Quick Sort

Pole: `[5, 3, 8, 1, 2]`, pivot = poslední prvek

1. úroveň: pivot = **2**

< pivot	pivot	> pivot
`[1]`	**2**	`[5, 3, 8]`

`[1]` je seřazené, řadíme `[5, 3, 8]`

Příklad – Quick Sort

2. úroveň: `[5, 3, 8]`, pivot = **8**

Příklad – Quick Sort

2. úroveň: `[5, 3, 8]`, pivot = **8**

$<$ pivot	pivot	$>$ pivot
`[5, 3]`	**8**	`[]`

Příklad – Quick Sort

2. úroveň: `[5, 3, 8]`, pivot = **8**

$<$ pivot	pivot	$>$ pivot
`[5, 3]`	**8**	`[]`

3. úroveň: `[5, 3]`, pivot = **3**

Příklad – Quick Sort

2. úroveň: `[5, 3, 8]`, pivot = **8**

< pivot	pivot	> pivot
`[5, 3]`	**8**	`[]`

3. úroveň: `[5, 3]`, pivot = **3**

< pivot	pivot	> pivot
`[]`	**3**	`[5]`

Volba pivotu

Volba pivotu

Volba pivotu **zásadně ovlivňuje výkon** quick sortu

Volba pivotu

Volba pivotu **zásadně ovlivňuje výkon** quick sortu

1. První/poslední prvek

- Jednoduché, ale **nevýhodné na předseřazených polích** $\rightarrow \mathcal{O}(n^2)$

Volba pivotu

Volba pivotu **zásadně ovlivňuje výkon** quick sortu

1. První/poslední prvek

- Jednoduché, ale **nevýhodné na předseřazených polích** $\rightarrow \mathcal{O}(n^2)$

2. Náhodný prvek

- Nejčastěji používaná metoda
- Nejhorší případ je stále $\mathcal{O}(n^2)$, ale je **velmi nepravděpodobný**

Volba pivotu

Co je medián?

Volba pivotu

Co je medián? Prostřední hodnota v seřazené posloupnosti

Například medián z $\{8, 2, 5\} =$

Volba pivotu

Co je medián? Prostřední hodnota v seřazené posloupnosti

Například medián z $\{8, 2, 5\} = 5$

Volba pivotu

Co je medián? Prostřední hodnota v seřazené posloupnosti

Například medián z $\{8, 2, 5\} = 5$

3. Medián tří

- Vyber první, prostřední a poslední prvek
- Jako pivot zvol jejich **medián**
- Lepší ochrana proti nejhoršímu případu

Vlastnosti

Složitost:

- Nejhorší případ:

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – špatná volba pivotu (vždy min/max)

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – špatná volba pivotu (vždy min/max)
- Průměrný případ:

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – špatná volba pivotu (vždy min/max)
- Průměrný případ: $\mathcal{O}(n \log n)$

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – špatná volba pivotu (vždy min/max)
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ:

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – špatná volba pivotu (vždy min/max)
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ: $\mathcal{O}(n \log n)$ – pivot vždy dělí pole na poloviny

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – špatná volba pivotu (vždy min/max)
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ: $\mathcal{O}(n \log n)$ – pivot vždy dělí pole na poloviny

Vlastnosti

- **Nestabilní**

Vlastnosti

- **Nestabilní** – prohazování přes pivot mění pořadí

Vlastnosti

- **Nestabilní** – prohazování přes pivot mění pořadí
- **In-place**

Vlastnosti

- **Nestabilní** – prohazování přes pivot mění pořadí
- **In-place** – $\mathcal{O}(\log n)$ paměti na zásobníku (rekurze)

Vlastnosti

- **Nestabilní** – prohazování přes pivot mění pořadí
- **In-place** – $\mathcal{O}(\log n)$ paměti na zásobníku (rekurze)
- V praxi **nejrychlejší** pro velká pole

Vlastnosti

- **Nestabilní** – prohazování přes pivot mění pořadí
- **In-place** – $\mathcal{O}(\log n)$ paměti na zásobníku (rekurze)
- V praxi **nejrychlejší** pro velká pole
- Funkce `qsort()` ve standardní knihovně C (`<stdlib.h>`)

Merge Sort

Princip

Princip

Merge Sort také používá strategii **rozděl a panuj**, ale jiným způsobem

Princip

Merge Sort také používá strategii **rozděl a panuj**, ale jiným způsobem

Postup:

1. **Rozděl** pole na dvě poloviny

Princip

Merge Sort také používá strategii **rozděl a panuj**, ale jiným způsobem

Postup:

1. **Rozděl** pole na dvě poloviny
2. **Rekurzivně** seřaď obě poloviny

Princip

Merge Sort také používá strategii **rozděl a panuj**, ale jiným způsobem

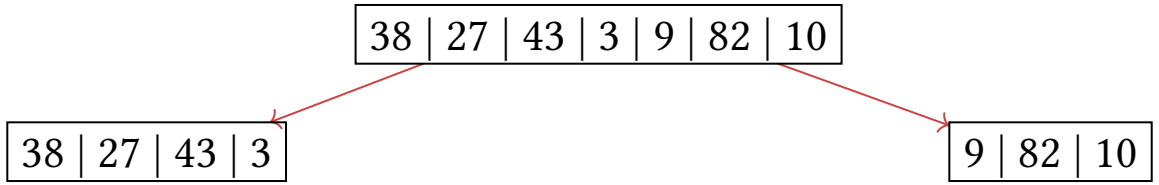
Postup:

1. **Rozděl** pole na dvě poloviny
2. **Rekurzivně** seřaď obě poloviny
3. **Spoj** (merge) obě seřazené poloviny do jednoho seřazeného pole

Vizualizace

38	27	43	3	9	82	10

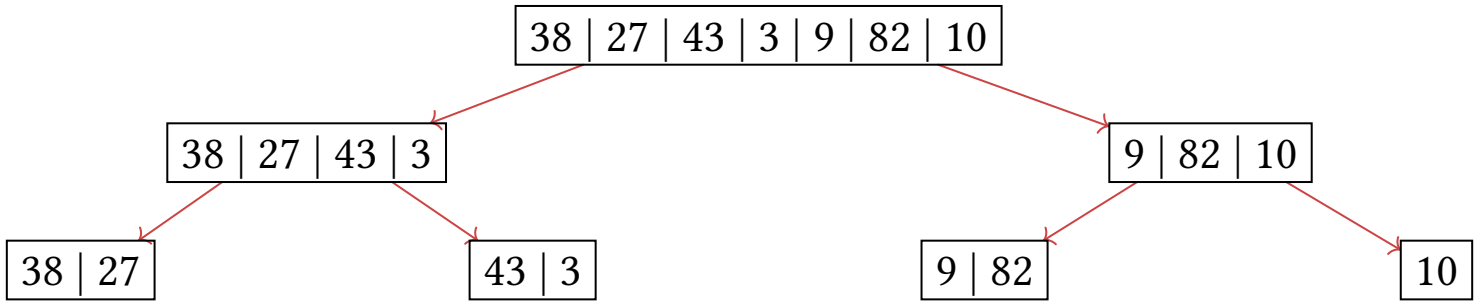
Vizualizace



rozděl

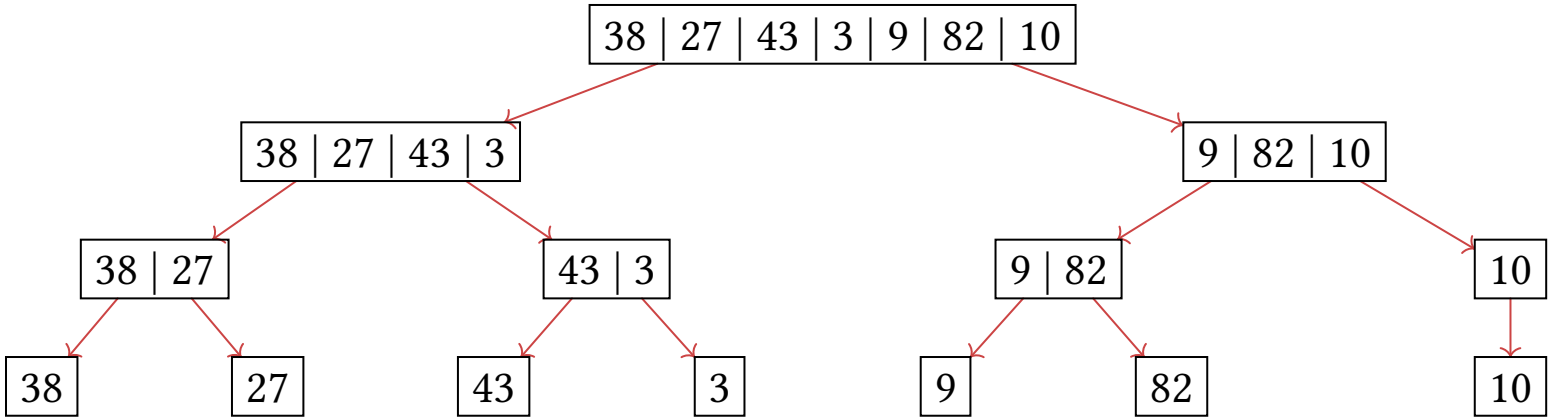
Vizualizace

rozděl

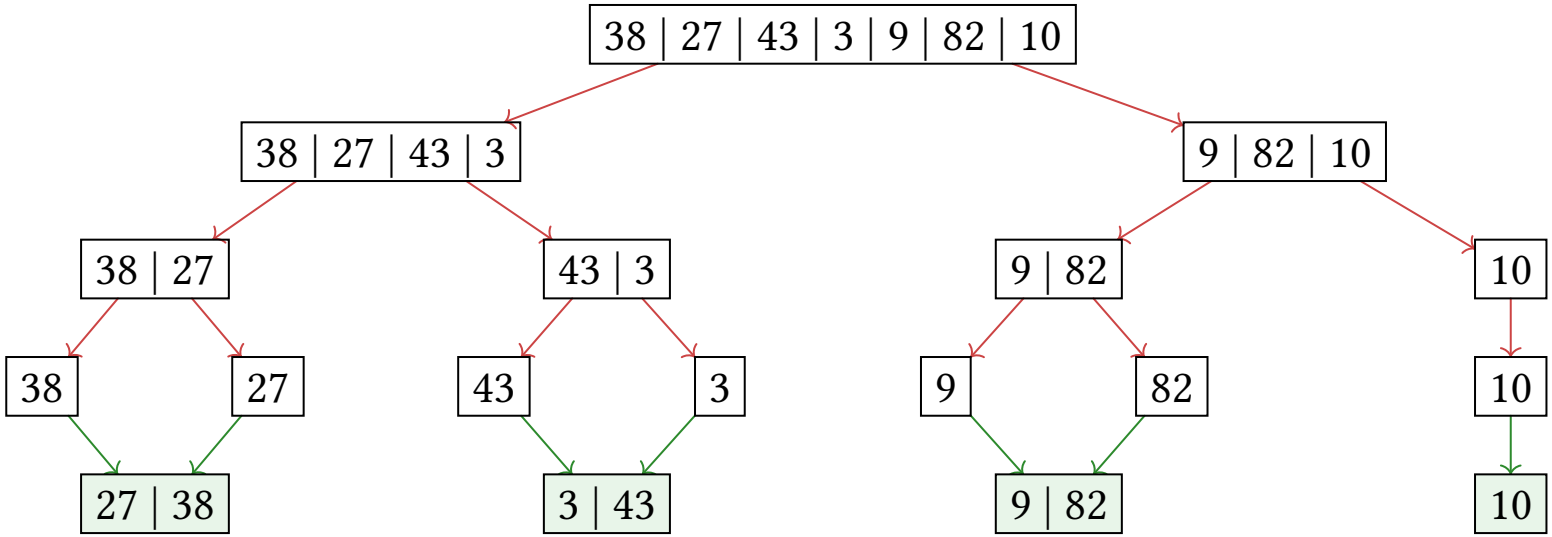


Vizualizace

rozděl

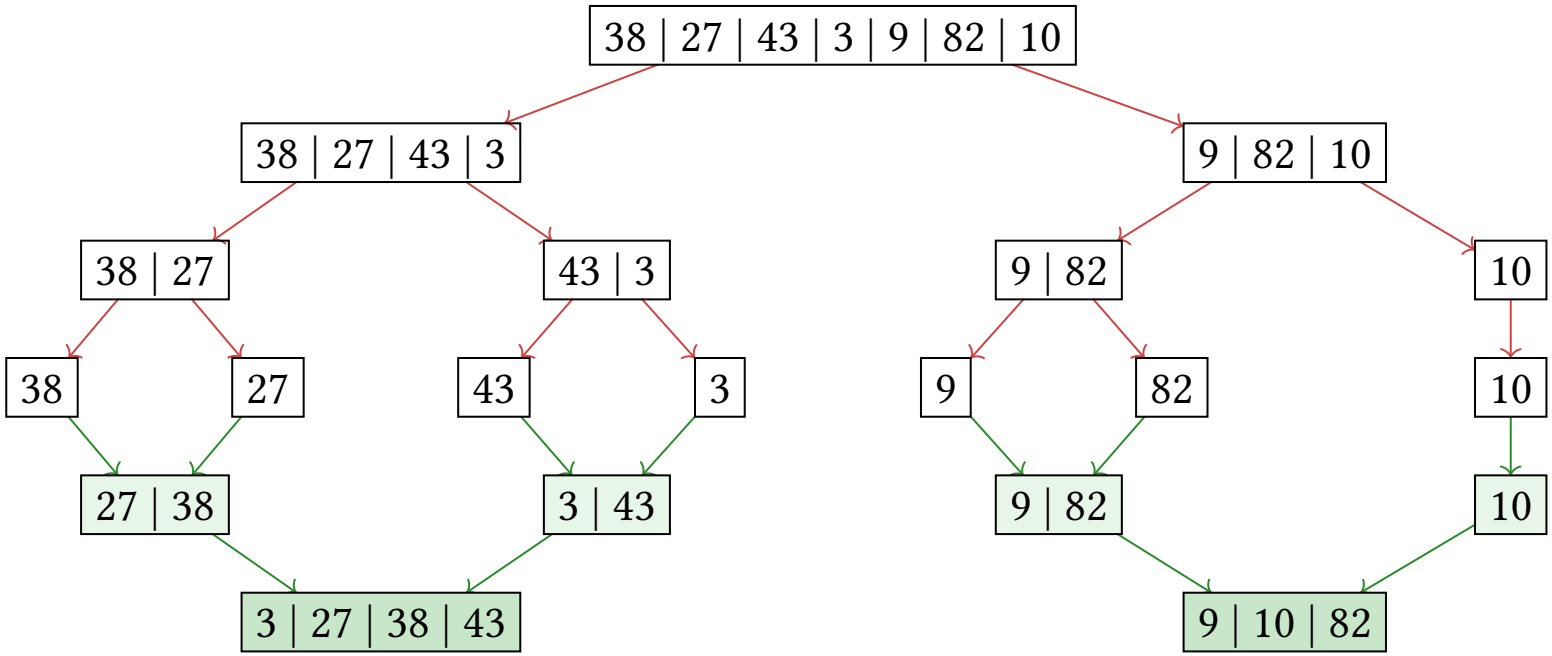


Vizualizace



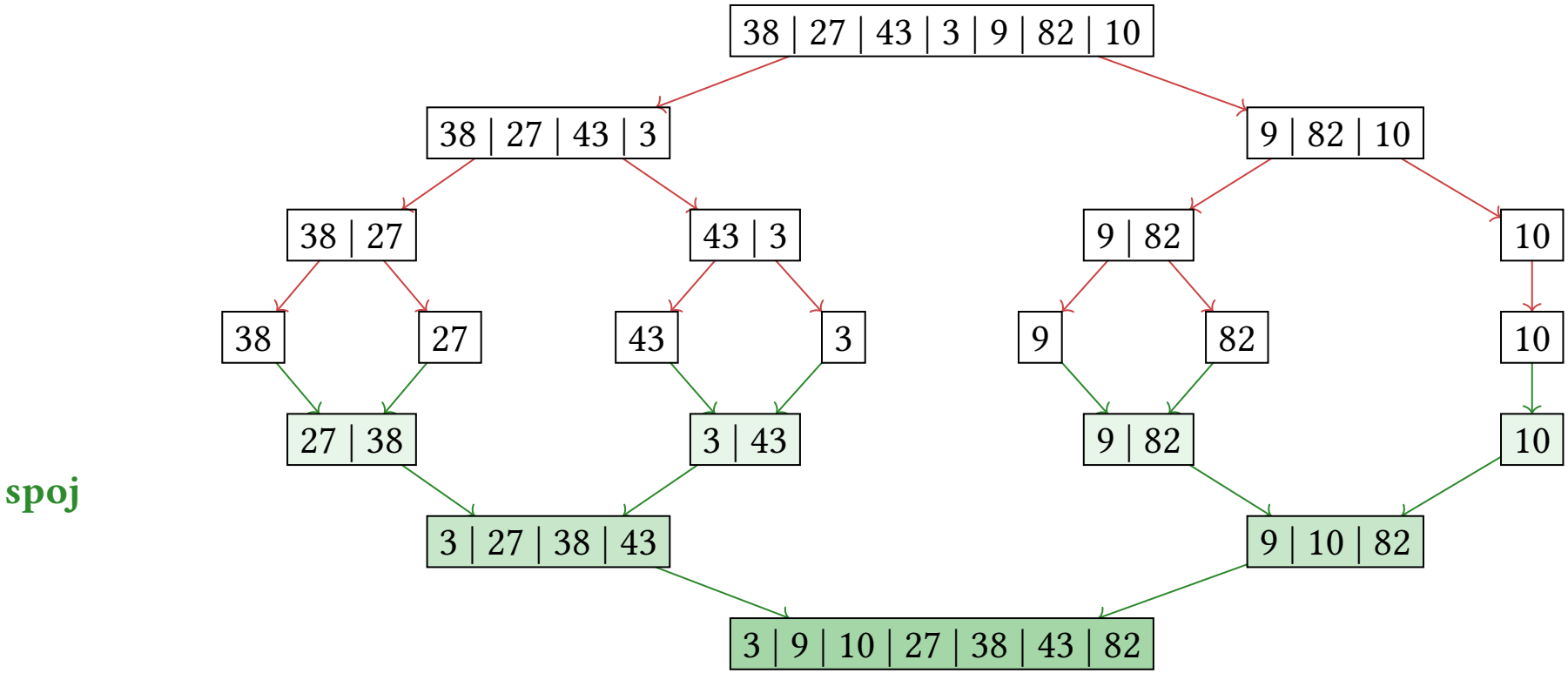
spoj

Vizualizace



spoj

Vizualizace



Vlastnosti

Složitost:

- Nejhorší případ:

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n \log n)$

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n \log n)$
- Průměrný případ:

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n \log n)$
- Průměrný případ: $\mathcal{O}(n \log n)$

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n \log n)$
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ:

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n \log n)$
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ: $\mathcal{O}(n \log n)$ – **vždy stejné!**

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n \log n)$
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ: $\mathcal{O}(n \log n)$ – **vždy stejné!**

Vlastnosti

- **Stabilní**

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků
- **Není in-place**

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků
- **Není in-place** – potřebuje $\mathcal{O}(n)$ extra paměti

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků
- **Není in-place** – potřebuje $\mathcal{O}(n)$ extra paměti
- **Garantovaná** složitost

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků
- **Není in-place** – potřebuje $\mathcal{O}(n)$ extra paměti
- **Garantovaná** složitost – žádný nejhorší případ jako u quick sortu

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků
- **Není in-place** – potřebuje $\mathcal{O}(n)$ extra paměti
- **Garantovaná** složitost – žádný nejhorší případ jako u quick sortu
- Vhodný pro **řazení dat ve spojovém seznamu**

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků
- **Není in-place** – potřebuje $\mathcal{O}(n)$ extra paměti
- **Garantovaná** složitost – žádný nejhorší případ jako u quick sortu
- Vhodný pro **řazení dat ve spojovém seznamu** – nepotřebuje random access, slévání funguje pouhým přepojováním `next` pointerů

Odvození asymptotické složitosti

Master theorem

Jak spočítáme složitost rekurzivního algoritmu?

Master theorem

Jak spočítáme složitost rekurzivního algoritmu?

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Master theorem

Jak spočítáme složitost rekurzivního algoritmu?

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

- a – počet podproblémů (na kolik částí dělíme)

Master theorem

Jak spočítáme složitost rekurzivního algoritmu?

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

- a – počet podproblémů (na kolik částí dělíme)
- b – faktor zmenšení (kolikrát je podproblém menší)

Master theorem

Jak spočítáme složitost rekurzivního algoritmu?

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

- a – počet podproblémů (na kolik částí dělíme)
- b – faktor zmenšení (kolikrát je podproblém menší)
- $f(n)$ – režie spojení výsledků (práce mimo rekurzi)

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$		

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	$T(n) = \Theta\left(n^{\log_b(a)}\right)$

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	$T(n) = \Theta\left(n^{\log_b(a)}\right)$
$f(n) = \Theta\left(n^{\log_b(a)}\right)$		

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	$T(n) = \Theta\left(n^{\log_b(a)}\right)$
$f(n) = \Theta\left(n^{\log_b(a)}\right)$	Vyrovnané	

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	$T(n) = \Theta\left(n^{\log_b(a)}\right)$
$f(n) = \Theta\left(n^{\log_b(a)}\right)$	Vyrovnané	$T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	$T(n) = \Theta\left(n^{\log_b(a)}\right)$
$f(n) = \Theta\left(n^{\log_b(a)}\right)$	Vyrovnané	$T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$
$f(n) = \Omega\left(n^{\log_b(a)+\varepsilon}\right)$		

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	$T(n) = \Theta\left(n^{\log_b(a)}\right)$
$f(n) = \Theta\left(n^{\log_b(a)}\right)$	Vyrovnané	$T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$
$f(n) = \Omega\left(n^{\log_b(a)+\varepsilon}\right)$	Spojení	

Master theorem

Podmínka	Dominuje	Výsledek
$f(n) = \mathcal{O}\left(n^{\log_b(a)-\varepsilon}\right)$	Rekurze	$T(n) = \Theta\left(n^{\log_b(a)}\right)$
$f(n) = \Theta\left(n^{\log_b(a)}\right)$	Vyrovnané	$T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$
$f(n) = \Omega\left(n^{\log_b(a)+\varepsilon}\right)$	Spojení	$T(n) = \Theta(f(n))$

Merge sort – odvození složitosti

Merge sort – odvození složitosti

Merge sort: $T(n) =$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) +$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a =$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2, b =$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) =$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(2) =$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(2) = 1$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(2) = 1$
- $f(n) =$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(2) = 1$
- $f(n) = \Theta(n^1) \Rightarrow$ **případ 2**

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(2) = 1$
- $f(n) = \Theta(n^1) \Rightarrow$ **případ 2**

Případ 2: $f(n) = \Theta\left(n^{\log_b(a)}\right) \rightarrow T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(2) = 1$
- $f(n) = \Theta(n^1) \Rightarrow$ **případ 2**

Případ 2: $f(n) = \Theta\left(n^{\log_b(a)}\right) \rightarrow T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$

A tedy výsledná složitost merge sortu je:

Merge sort – odvození složitosti

Merge sort: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 2$, $b = 2$, $f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(2) = 1$
- $f(n) = \Theta(n^1) \Rightarrow$ **případ 2**

Případ 2: $f(n) = \Theta\left(n^{\log_b(a)}\right) \rightarrow T(n) = \Theta\left(n^{\log_b(a)} \cdot \log n\right)$

A tedy výsledná složitost merge sortu je:

$$T(n) = \Theta(n \cdot \log n)$$

Násobení dvou čísel

Násobení dvou n-ciferných čísel

Násobení dvou n-ciferných čísel

Mějme dvě n-ciferná čísla x a y, která chceme vynásobit (n je mocnina dvou)

Násobení dvou n-ciferných čísel

Mějme dvě n-ciferná čísla x a y, která chceme vynásobit (n je mocnina dvou)

Obě čísla rozdělíme na horních $\frac{n}{2}$ a dolních $\frac{n}{2}$ cifer:

Násobení dvou n-ciferných čísel

Mějme dvě n-ciferná čísla x a y, která chceme vynásobit (n je mocnina dvou)

Obě čísla rozdělíme na horních $\frac{n}{2}$ a dolních $\frac{n}{2}$ cifer:

$$x = x_U \cdot 10^{\frac{n}{2}} + x_L$$

$$y = y_U \cdot 10^{\frac{n}{2}} + y_L$$

Násobení dvou n-ciferných čísel

Výsledný součin lze poskládat ze **4 součinů** $\frac{n}{2}$-ciferných čísel:

Násobení dvou n-ciferných čísel

Výsledný součin lze poskládat ze **4 součinů** $\frac{n}{2}$-ciferných čísel:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

Násobení dvou n-ciferných čísel

Výsledný součin lze poskládat ze **4 součinů** $\frac{n}{2}$-ciferných čísel:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

Jakou asymptotickou složitost má tento styl násobení?

Násobení dvou n-ciferných čísel

Výsledný součin lze poskládat ze **4 součinů** $\frac{n}{2}$-ciferných čísel:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

Jakou asymptotickou složitost má tento styl násobení?

$T(n) = 4 \cdot T(\frac{n}{2}) + \mathcal{O}(n)$

Násobení dvou n-ciferných čísel

Výsledný součin lze poskládat ze **4 součinů** $\frac{n}{2}$-ciferných čísel:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

Jakou asymptotickou složitost má tento styl násobení?

$T(n) = 4 \cdot T(\frac{n}{2}) + \mathcal{O}(n) \Rightarrow \log_2(4) = 2$

Násobení dvou n-ciferných čísel

Výsledný součin lze poskládat ze **4 součinů** $\frac{n}{2}$-ciferných čísel:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

Jakou asymptotickou složitost má tento styl násobení?

$T(n) = 4 \cdot T(\frac{n}{2}) + \mathcal{O}(n) \Rightarrow \log_2(4) = 2 \Rightarrow$ případ 1

Násobení dvou n-ciferných čísel

Výsledný součin lze poskládat ze **4 součinů** $\frac{n}{2}$-ciferných čísel:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

Jakou asymptotickou složitost má tento styl násobení?

$T(n) = 4 \cdot T(\frac{n}{2}) + \mathcal{O}(n) \Rightarrow \log_2(4) = 2 \Rightarrow$ případ 1 $\Rightarrow \mathcal{O}(n^2)$

Karacubovo násobení

Karacubovo násobení

Vylepšení – snížíme počet násobení ze **4 na 3**:

Karacubovo násobení

Vylepšení – snížíme počet násobení ze **4 na 3**:

$$x_U \cdot y_L + x_L \cdot y_U = (x_U + x_L) \cdot (y_U + y_L) - x_U \cdot y_U - x_L \cdot y_L$$

Karacubovo násobení

Důkaz:

Karacubovo násobení

Důkaz:

$$(x_U + x_L) \cdot (y_U + y_L) - x_U \cdot y_U - x_L \cdot y_L$$

Karacubovo násobení

Důkaz:

$$(x_U + x_L) \cdot (y_U + y_L) - x_U \cdot y_U - x_L \cdot y_L$$

$$= x_U \cdot y_U + x_U \cdot y_L + x_L \cdot y_U + x_L \cdot y_L - x_U \cdot y_U - x_L \cdot y_L$$

Karacubovo násobení

Důkaz:

$$(x_U + x_L) \cdot (y_U + y_L) - x_U \cdot y_U - x_L \cdot y_L$$

$$= x_U \cdot y_U + x_U \cdot y_L + x_L \cdot y_U + x_L \cdot y_L - x_U \cdot y_U - x_L \cdot y_L$$

$$= x_U \cdot y_L + x_L \cdot y_U$$

Karacubovo násobení

Dosadíme vylepšení do původního vzorce:

Karacubovo násobení

Dosadíme vylepšení do původního vzorce:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

Karacubovo násobení

Dosadíme vylepšení do původního vzorce:

$$x \cdot y = x_U \cdot y_U \cdot 10^n + (x_U \cdot y_L + x_L \cdot y_U) \cdot 10^{\frac{n}{2}} + x_L \cdot y_L$$

$$= \underbrace{x_U \cdot y_U}_{\text{a}} \cdot 10^n + \underbrace{x_L \cdot y_L}_{\text{b}}$$

$$+ \left(\underbrace{(x_U + x_L) \cdot (y_U + y_L)}_{\text{c}} - \underbrace{x_U \cdot y_U}_{\text{a}} - \underbrace{x_L \cdot y_L}_{\text{b}} \right) \cdot 10^{\frac{n}{2}}$$

Karacubovo násobení

Karacubovo násobení používá pouze **3 rekurzivní násobení**:

Karacubovo násobení

Karacubovo násobení používá pouze **3 rekurzivní násobení**:

- $x_U \cdot y_U$

Karacubovo násobení

Karacubovo násobení používá pouze **3 rekurzivní násobení**:

- $x_U \cdot y_U$
- $x_L \cdot y_L$

Karacubovo násobení

Karacubovo násobení používá pouze **3 rekurzivní násobení**:

- $x_U \cdot y_U$
- $x_L \cdot y_L$
- $(x_U + x_L) \cdot (y_U + y_L)$

Karacubovo násobení

Karacubovo násobení používá pouze **3 rekurzivní násobení**:

- $x_U \cdot y_U$
- $x_L \cdot y_L$
- $(x_U + x_L) \cdot (y_U + y_L)$

Ostatní operace (sčítání, odčítání, násobení 10^n) jsou $\mathcal{O}(n)$

Karacubovo násobení

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

Karacubovo násobení

$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 3, b = 2, f(n) = \mathcal{O}(n)$

Karacubovo násobení

$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 3, b = 2, f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(3) \approx 1.585$

Karacubovo násobení

$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 3, b = 2, f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(3) \approx 1.585$
- $f(n) = \mathcal{O}(n^1) = \mathcal{O}(n^{1.585-0.585}) \Rightarrow$ **případ 1**

Karacubovo násobení

$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 3, b = 2, f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(3) \approx 1.585$
- $f(n) = \mathcal{O}(n^1) = \mathcal{O}(n^{1.585-0.585}) \Rightarrow$ **případ 1**

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \mathcal{O}(n^{1.585})$$

Karacubovo násobení

$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \mathcal{O}(n)$

- $a = 3, b = 2, f(n) = \mathcal{O}(n)$
- $\log_b(a) = \log_2(3) \approx 1.585$
- $f(n) = \mathcal{O}(n^1) = \mathcal{O}(n^{1.585-0.585}) \Rightarrow$ **případ 1**

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \mathcal{O}(n^{1.585})$$

Výrazné zlepšení oproti $\mathcal{O}(n^2)$!

Příklad – 12×34

$x = 12, y = 34$

Příklad – 12×34

$x = 12, y = 34 \Rightarrow x_U = 1, x_L = 2, y_U = 3, y_L = 4$

Příklad – 12×34

$x = 12, y = 34 \Rightarrow x_U = 1, x_L = 2, y_U = 3, y_L = 4$

Tři násobení:

- $x_U \cdot y_U = 1 \cdot 3 = 3$

Příklad – 12×34

$x = 12, y = 34 \Rightarrow x_U = 1, x_L = 2, y_U = 3, y_L = 4$

Tři násobení:

- $x_U \cdot y_U = 1 \cdot 3 = 3$
- $x_L \cdot y_L = 2 \cdot 4 = 8$

Příklad – 12×34

$x = 12, y = 34 \Rightarrow x_U = 1, x_L = 2, y_U = 3, y_L = 4$

Tři násobení:

- $x_U \cdot y_U = 1 \cdot 3 = 3$
- $x_L \cdot y_L = 2 \cdot 4 = 8$
- $(x_U + x_L) \cdot (y_U + y_L) = 3 \cdot 7 = 21$

Příklad – 12×34

$x = 12, y = 34 \Rightarrow x_U = 1, x_L = 2, y_U = 3, y_L = 4$

Tři násobení:

- $x_U \cdot y_U = 1 \cdot 3 = 3$
- $x_L \cdot y_L = 2 \cdot 4 = 8$
- $(x_U + x_L) \cdot (y_U + y_L) = 3 \cdot 7 = 21$

Dosadíme:

$$3 \cdot 10^2 + (21 - 3 - 8) \cdot 10^1 + 8 = 300 + 100 + 8 = 408$$

Příklad – 12×34

$x = 12, y = 34 \Rightarrow x_U = 1, x_L = 2, y_U = 3, y_L = 4$

Tři násobení:

- $x_U \cdot y_U = 1 \cdot 3 = 3$
- $x_L \cdot y_L = 2 \cdot 4 = 8$
- $(x_U + x_L) \cdot (y_U + y_L) = 3 \cdot 7 = 21$

Dosadíme:

$$3 \cdot 10^2 + (21 - 3 - 8) \cdot 10^1 + 8 = 300 + 100 + 8 = 408 = 12 \times 34$$

Srovnání

	Nejlepší	Průměr	Nejhorší	Paměť	Stab.	In-pl.
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	Ne	Ano

	Nejlepší	Průměr	Nejhorší	Paměť	Stab.	In-pl.
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	Ne	Ano
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Ano	Ne

Shrnutí

Shrnutí

- **Rozděl a panuj** – rozdělíme problém na menší části, vyřešíme rekurzivně, spojíme výsledky

Shrnutí

- **Rozděl a panuj** – rozdělíme problém na menší části, vyřešíme rekurzivně, spojíme výsledky
- **Master theorem** – nástroj pro odvození složitosti rekurzivních algoritmů

Shrnutí

- **Rozděl a panuj** – rozdělíme problém na menší části, vyřešíme rekurzivně, spojíme výsledky
- **Master theorem** – nástroj pro odvození složitosti rekurzivních algoritmů
- **Quick Sort** – v praxi nejrychlejší, ale v nejhorším případě $\mathcal{O}(n^2)$; volba pivotu je klíčová

Shrnutí

- **Rozděl a panuj** – rozdělíme problém na menší části, vyřešíme rekurzivně, spojíme výsledky
- **Master theorem** – nástroj pro odvození složitosti rekurzivních algoritmů
- **Quick Sort** – v praxi nejrychlejší, ale v nejhorším případě $\mathcal{O}(n^2)$; volba pivotu je klíčová
- **Merge Sort** – garantovaný $\mathcal{O}(n \log n)$, stabilní, ale $\mathcal{O}(n)$ extra paměti

Shrnutí

- **Rozděl a panuj** – rozdělíme problém na menší části, vyřešíme rekurzivně, spojíme výsledky
- **Master theorem** – nástroj pro odvození složitosti rekurzivních algoritmů
- **Quick Sort** – v praxi nejrychlejší, ale v nejhorším případě $\mathcal{O}(n^2)$; volba pivotu je klíčová
- **Merge Sort** – garantovaný $\mathcal{O}(n \log n)$, stabilní, ale $\mathcal{O}(n)$ extra paměti
- **Karacubovo násobení** – ukázka, že rozděl a panuj může zlepšit i "jednoduchou" operaci ($\mathcal{O}(n^2) \to \mathcal{O}(n^{1.585})$)